

Semi-Custodial Wallet : Enterprise ready Wallet SDK

Project Name: Hodei Wallet

Version: 1.3

Last Update: 2026-05-11

Current Milestone: 4

Catalyst Link: <https://milestones.projectcatalyst.io/projects/1400011>

Public Github Repository: https://github.com/Cardano-Forge/hodei_wallet_f14

Approvals:

Team	Approved	Date
Development Team	Dodilanne	2026-01-20
Project Manager	Mr. Abdibdi	2026-01-20
Stakeholders	Tqueri	2026-01-20
Management	Zachary Soesbee	2026-01-20

Changelog

Date	Comment
2026-01-20	More information about Authentication Partner(s) and Frontend Integration . (Architecture Decision) Added Preliminary Events . (Telemetry)
2026-03-11	Added #Milestone 2 - https://milestones.projectcatalyst.io/projects/1400011/milestones/2 Landing Page: https://hodei.io APK: https://hodei-f14.s3.ca-central-1.amazonaws.com/hodei-vault-latest.apk Apple: https://testflight.apple.com/join/JuGKzBzH
2026-03-19	Added Screenshots to show the date synced using Apple iCloud, iPhone wallet created on device and data is accessible on MacOS keychain.
2026-04-17	Added Milestone 3 - https://milestones.projectcatalyst.io/projects/1400011/milestones/3 Pending approval for PIA document
2026-05-11	Added missing proofs, urls, videos. See Milestone 3 for details.

Table of Content

- System Overview..... 4
 - Components..... 5
 - Communication Protocol..... 5
 - Multi-Link Support..... 5
 - Platform Considerations..... 5
 - Flows..... 6
 - Wallet..... 6
 - Linking Flow..... 6
 - Unlinking Flow..... 6
 - Linking methods..... 6
 - Transaction Signing Flow..... 7
- Data Classification..... 8
 - Sensitivity Levels..... 8
 - Seed Phrase Management..... 8
 - Bridge Database..... 8
 - Telemetry..... 8
 - Data Inventory..... 9
 - Key Management..... 10
 - Key Generation..... 10
 - Key Storage..... 10
 - Key Usage..... 10
 - Memory lifecycle..... 10
 - Vault Structure..... 11
 - Import & Export..... 12
 - Import Seed Phrase..... 12
 - Export Seed Phrase (Reveal)..... 13
 - Key Backup & Recovery..... 14
 - Recovery Flow..... 14
 - Key Destruction..... 14
 - Delete Single Seed Flow..... 14
 - Delete All Data Flow..... 15
 - Key Lost / Compromised..... 15
- Telemetry & Privacy..... 16
 - Principles..... 16
 - What is Collected ?..... 16
 - Retention Policy..... 16
- Threats and Security Review..... 17
 - Threats..... 17
 - Severity Legend..... 17
 - Platform Security Parity Notice..... 17
 - List of Identified Threats..... 17
 - Private Key Exfiltration..... 18
 - Unauthorized Transaction Signing..... 19
 - Telemetry Data Leakage..... 20
 - Cloud Storage Compromise..... 21
 - Man-in-the-Middle Attacks..... 22

Private Key Extraction (Technical Methods).....	23
Seed Phrase Compromise Through Telemetry or Logging.....	24
Transaction Tampering or Replay Attacks.....	25
Phishing Attacks Mimicking Wallet Interface.....	26
Supply Chain Compromise of Wallet Dependencies.....	27
Cloud Backup Failure.....	28
Seed Phrase Theft During Recovery.....	29
Trust Boundaries.....	30
Component Clarification.....	30
Boundary Definitions.....	30
What Each Zone Trusts.....	30
Critical Trust Boundary: User Review.....	30
Risk Matrix.....	31
Priority Definitions.....	31
Threat Summary.....	31
Accepted Risks.....	31
Architecture Decisions.....	32
Vault.....	32
Bridge.....	32
Client.....	32
Milestone 2.....	33
Manual tests results.....	33
Automated test results on Android.....	34
Live Demos.....	34
Links to Video and Submitted Transaction.....	34
Milestone 3.....	36
Privacy impact assessment (TBC).....	36
Proofs.....	36
Telemetry Update.....	36
Infrastructure.....	37
Test Report Milestone 3.....	37
Changelog.....	39

System Overview

This solution is composed of 3 components, the ***Vault***, the ***Bridge*** and the ***Client*** Connector.



Hodei Wallet is designed for easy setup without complex seed phrase management. Built on a 3-component architecture, it integrates seamlessly with existing dApps and wallet connectors without requiring major changes.

Private keys are stored securely on the user's device using built-in secure storage (like *Apple's iCloud Keychain*) and backed up to the user's cloud when available, so only they can access them.

The keys remain isolated from dApps, as all communication is routed through a secure bridge. Users can then review and sign transactions directly from their mobile device using their device password or biometrics.

Components

Component	Description	Private Data
Vault	Native app (iOS, Android). Holds private keys in platform secure storage. Signs transactions.	Yes
Bridge	Stateful relay with KV Database. Stores unsigned TX temporarily, relay signature to frontend using Web Socket. Manages Session Tokens.	No
Client	Web Component/JS library embedded in third-party apps. CIP-30 fully compliant, drop-in replacement for browser extensions.	No

Communication Protocol

For this concept, we are going to implement Web Sockets between the ***Vault*** < > ***Bridge*** < > ***Client***.

Multi-Link Support

- One ***Vault*** can link to multiple ***Clients***
- dApps do not have visibility into other linked apps
- A ***Vault*** can view and manage linked ***Clients***.

Platform Considerations

Platform	Key Storage	Status
iOS	iCloud Keychain	In scope
Android	Encrypted Shared Preferences	In scope

Flows

Wallet

Linking Flow

1. User clicks "Connect Wallet" in **Client**
↓
2. **Client** requests linking session from **Bridge**
↓
3. **Bridge** generates PIN + session ID, returns to **Client**
↓
4. Web Component displays PIN on screen
↓
5. User opens **Vault**, taps "Link"
↓
6. User enters PIN manually
↓
7. **Vault** sends PIN + **Vault** ID + signature (to validate wallet ownership) to **Bridge**
↓
8. **Bridge** validates PIN, generates Session Token
↓
9. **Bridge** returns Session to **Client**

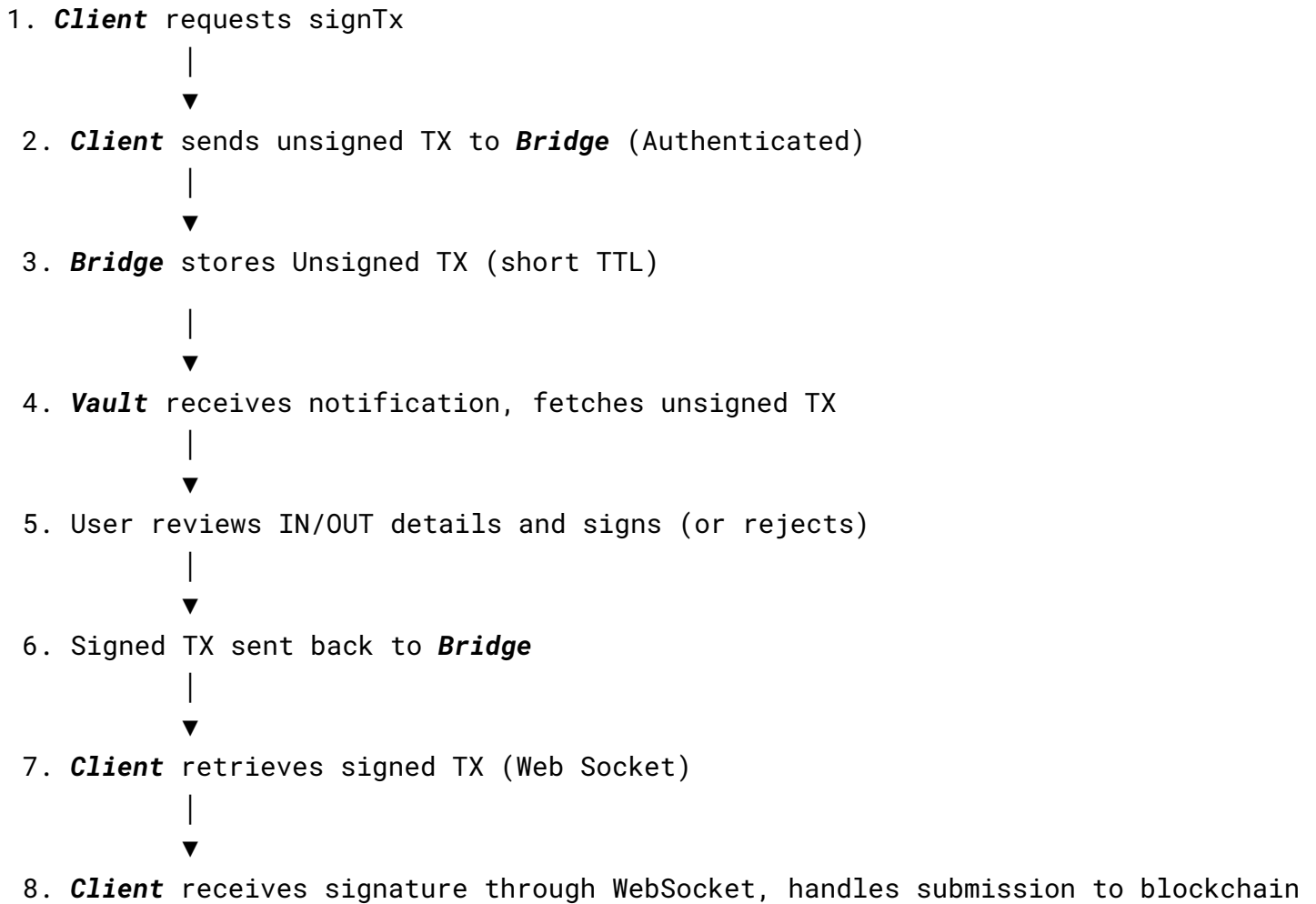
Unlinking Flow

1. User initiates unlink (from **Vault or Client**)
↓
2. Authenticated Request sent to **Bridge**
↓
3. **Bridge** removes device-app link
↓
4. **Bridge** notifies and closes active **Clients/Vault**
↓
5. Both sides cleanup

Linking methods

- **Manual PIN entry:** Fallback for same-device scenarios (e.g., mobile browser + mobile app)

Transaction Signing Flow



Note: The **Bridge** does NOT submit transactions. Submission is always the **Client**'s responsibility.

Data Classification

Sensitivity Levels

Level	Definition	Examples
Critical	Compromise leads to total loss of funds	Private keys, seed phrase
High	Compromise enables unauthorized actions	Session Token (can request signatures)
Medium	Exposure reveals transaction intent, no direct fund loss	Unsigned/signed TX
Low	Minimal impact, no financial risk	Device IDs, app metadata

Seed Phrase Management

Default behavior: Seed phrase is stored in platform keychain (Synced to cloud provider keychain when available). This approach targets non-crypto-enthusiast and non-tech-savvy users who benefit from automatic, secure backup without manual steps (When Available).

Export option: Users can export their seed phrase manually (e.g., write on paper). The wallet must display a confirmation alert warning that revealing the seed phrase is unsafe and should only be done for personal backup purposes.

Bridge Database

Storage: The recommended database is a KV Store.

Encryption at rest: Optional. Transaction data originates from and is submitted to a public blockchain, so encryption at rest provides limited additional security.

TTL and cleanup: Unsigned transactions should have a short TTL to minimize data retention.

Signatures and Retries: No retry is available for signed transactions as the client disconnected and the dApp will not necessarily keep/reconnect the hooks to process the signature.

Telemetry

Approach: Privacy-first. No PII stored.

Goals: Prevent abuse, improve our software and related services and to make design decisions for future releases.

Guidelines:

- No PII in telemetry data
- Hash IP addresses for rate limiting and attack detection (*Using selected cloud Provided network stack*)
- Define retention policies (TBD)

Data Inventory

Data	Location	Sensitivity	Encrypted at Rest	Encrypted in Transit
Private Keys	Vault (native keychain)	Critical	Yes (platform managed)	N/A (never leaves device)
Seed Phrase	Vault (native keychain)	Critical	Yes (platform managed)	N/A (never leaves device)
Seed Backup	Cloud (iCloud Keychain)	Critical	Yes (platform managed)	Yes (TLS, platform managed)
Session Token	Client(browser/app)	High	Platform dependent	Yes (TLS)
Unsigned TX	Bridge (database, short TTL)	Medium	Optional (see notes)	Yes (TLS)
Signed TX	Bridge (Relayed)	Medium	Optional (see notes)	Yes (TLS)
Device ID	Bridge	Low	No	Yes (TLS)
Linked Apps List	Bridge	Low	Platform dependent	N/A
Telemetry	Telemetry backend	Low	Recommended	Yes (TLS)

Key Management

Key Generation

Aspect	Requirement	Owner
Library	CSL (Cardano Serialization Library) or equivalent	Mobile Team
Seed phrase standard	BIP-39 mnemonic (24 words recommended)	Mobile Team
Key derivation	BIP-32/BIP-44 hierarchical deterministic (HD) wallet	Mobile Team
Cardano-specific	CIP-1852 derivation path for Cardano	Mobile Team

Key Storage

Aspect	Requirement	Owner
iOS	Keychain	Mobile Team
Android	Android Encrypted Shared Preferences	Mobile Team

Key Usage

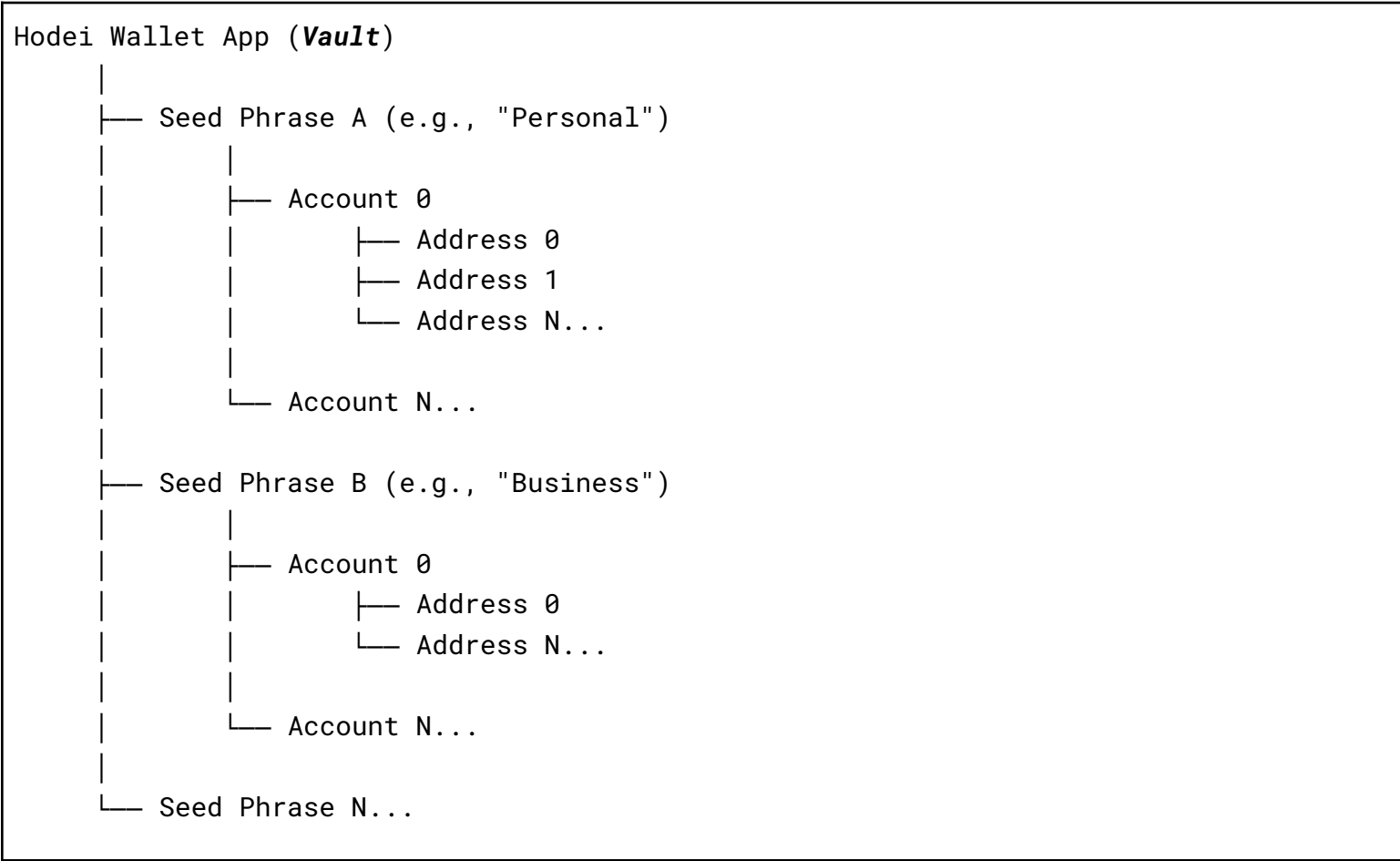
Aspect	Requirement	Owner
Access control	Biometric or PIN required before any key operation	Mobile Team
Memory handling	Load secrets only when needed, clear immediately after	Mobile Team
Key operations	Secrets used only for: wallet/account generation, TX signing, seed reveal	Mobile Team
No export	Private key never exported (only seed phrase can be revealed)	Mobile Team

Memory lifecycle

Operation	Secrets in memory	After operation
Wallet/account generation	Yes	Clear secrets, keep derived addresses only
Transaction signing	Yes	Clear secrets immediately after signing
Seed phrase reveal	Yes	Clear secrets after user dismisses screen
Normal wallet usage	No	Only public info (addresses, balances) in memory

Vault Structure

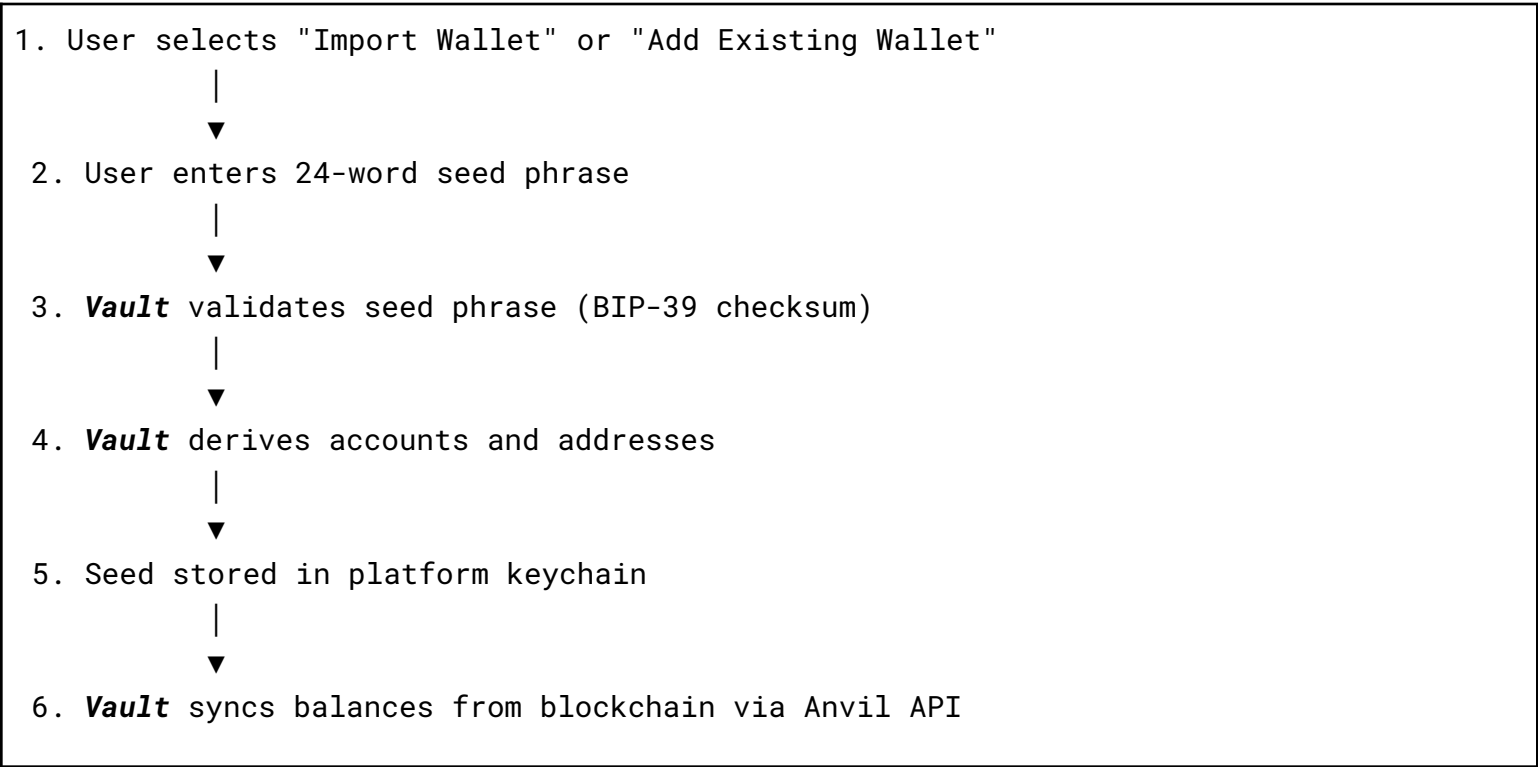
The **Vault** supports multiple seed phrases, each with hierarchical account management via HD derivation:



Feature	Description
Multi-seed	Users can add multiple seed phrases (create new or import existing)
Multi-account	Each seed can have multiple accounts
Multi-address	Each account can derive multiple addresses (per CIP-1852)
Independent backup	Each seed phrase has its own backup; must be backed up separately

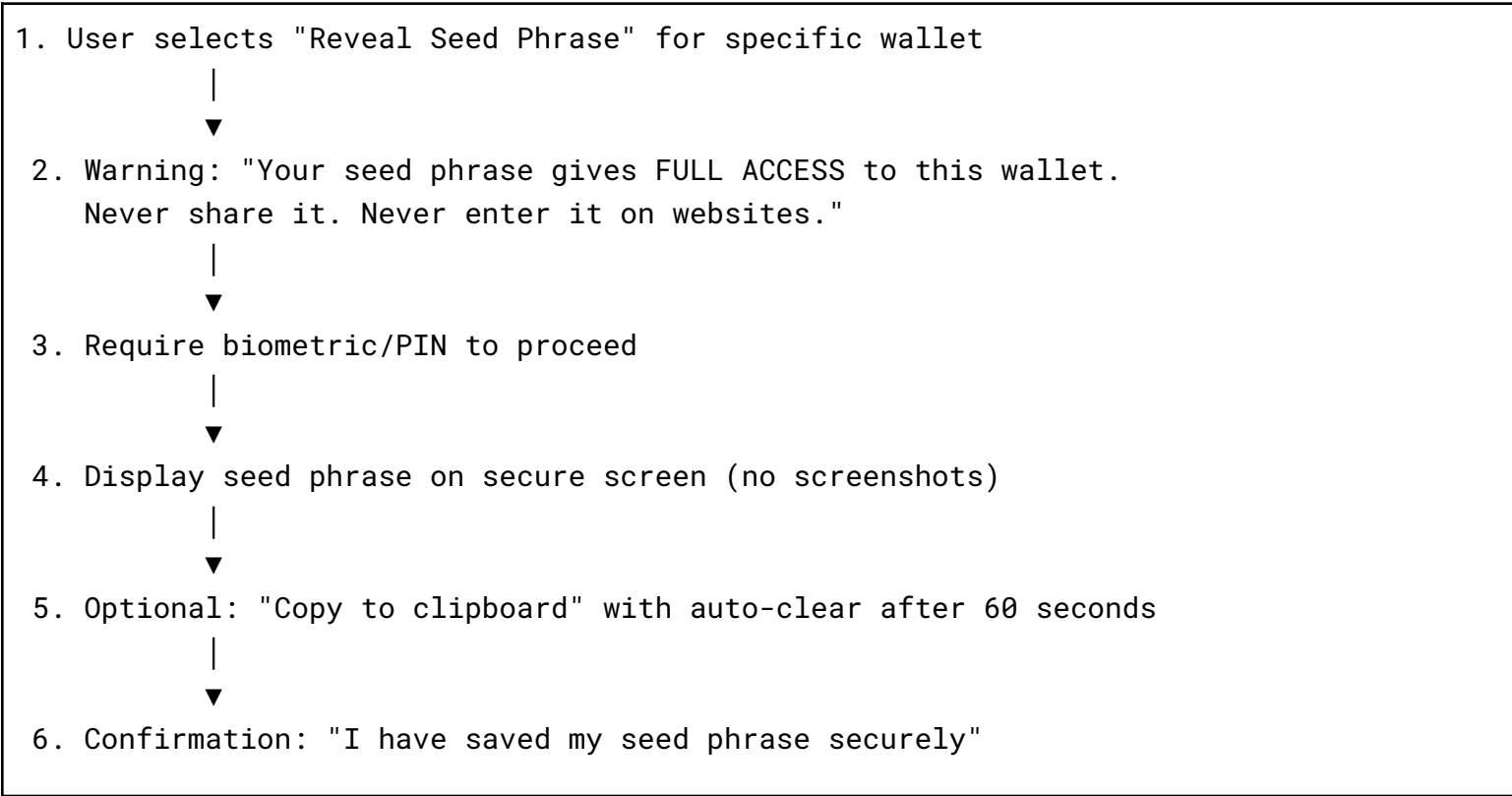
Import & Export

Import Seed Phrase



Aspect	Requirement	Owner
Validation	Verify BIP-39 checksum before accepting	Mobile Team
Secure input	Prevent screenshots during seed entry	Mobile Team
No clipboard	Warn user if pasting (clipboard may be insecure)	Mobile Team

Export Seed Phrase (Reveal)

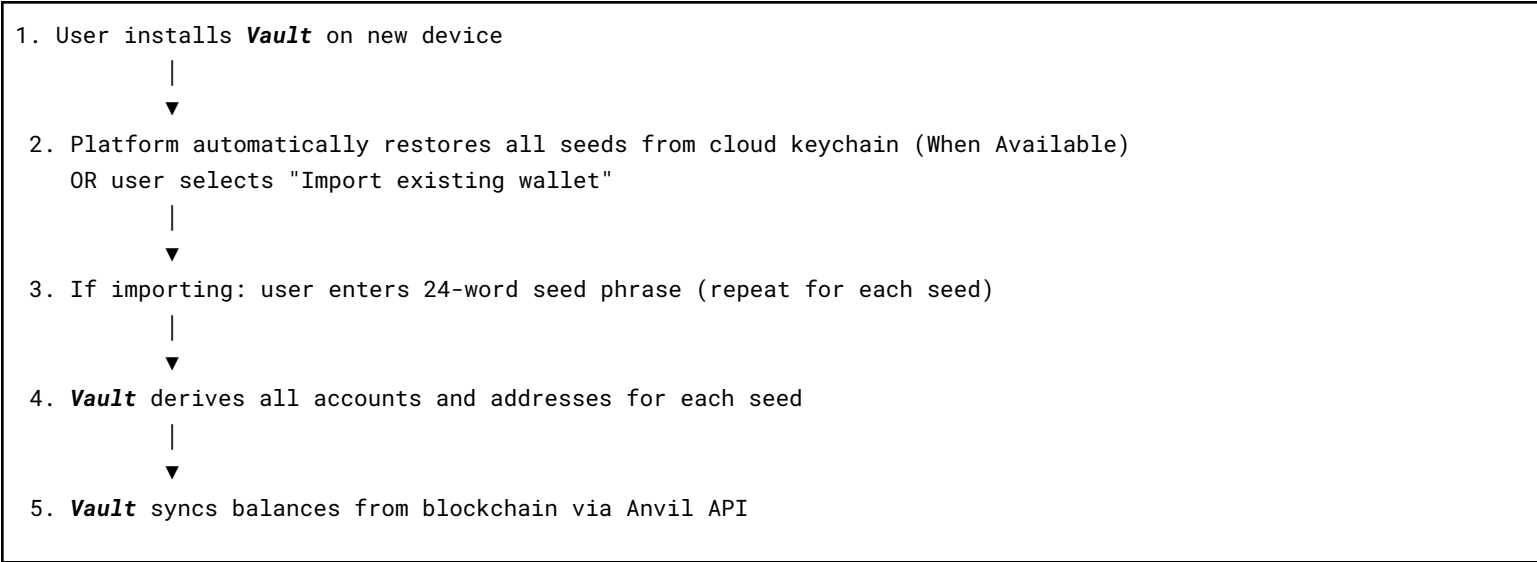


Aspect	Requirement	Owner
Warning	Clear warning about risks before revealing	Mobile Team
Authentication	Biometric/PIN required	Mobile Team
Secure display	Prevent screenshots/screen recording	Mobile Team
Clipboard	If allowed, auto-clear clipboard after short timeout	Mobile Team

Key Backup & Recovery

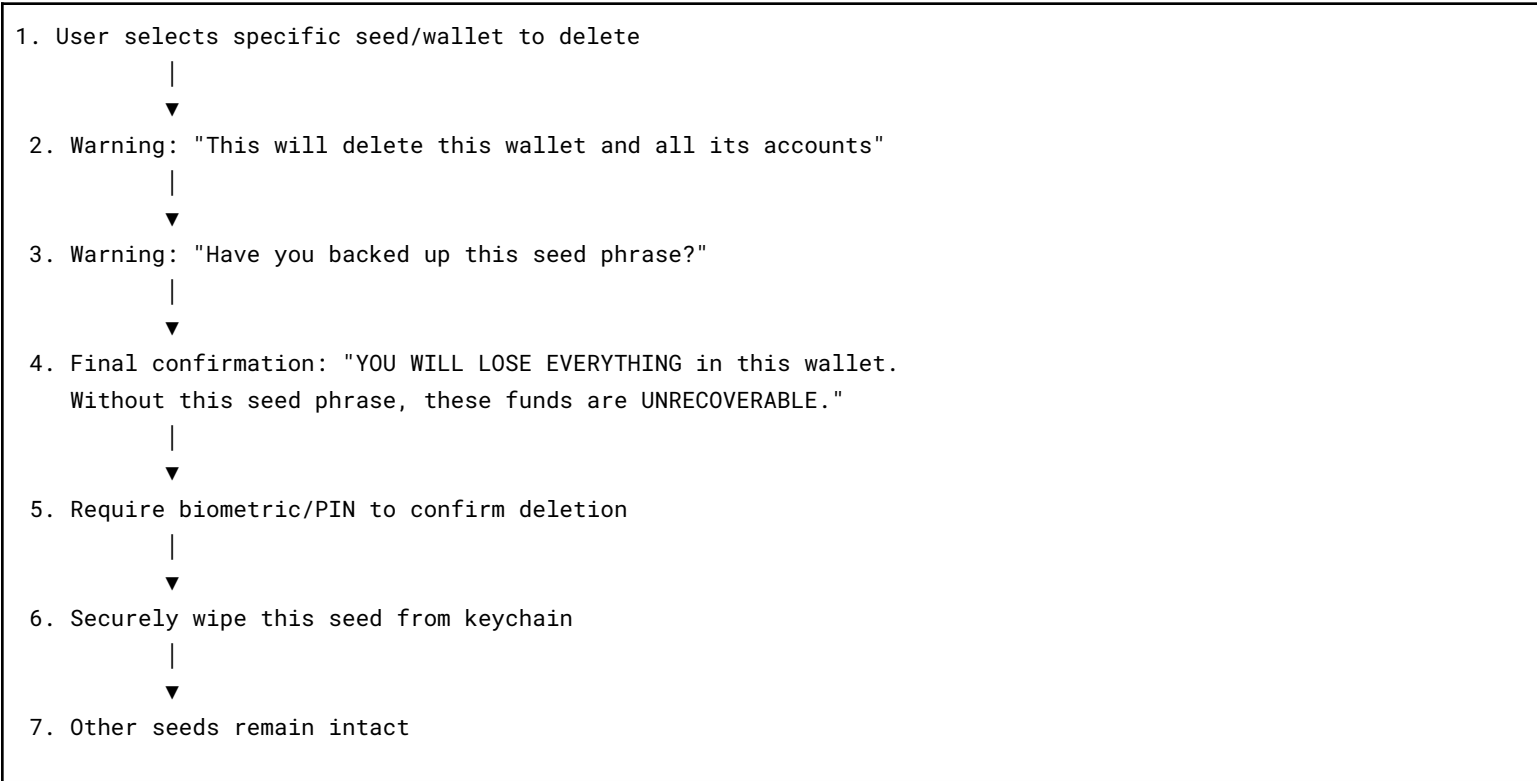
Aspect	Requirement	Owner
Manual backup	User can reveal/export each seed phrase separately (with warning)	Mobile Team
Recovery	Restore from platform backup (When Available) on new device, or import seed phrases	Mobile Team

Recovery Flow

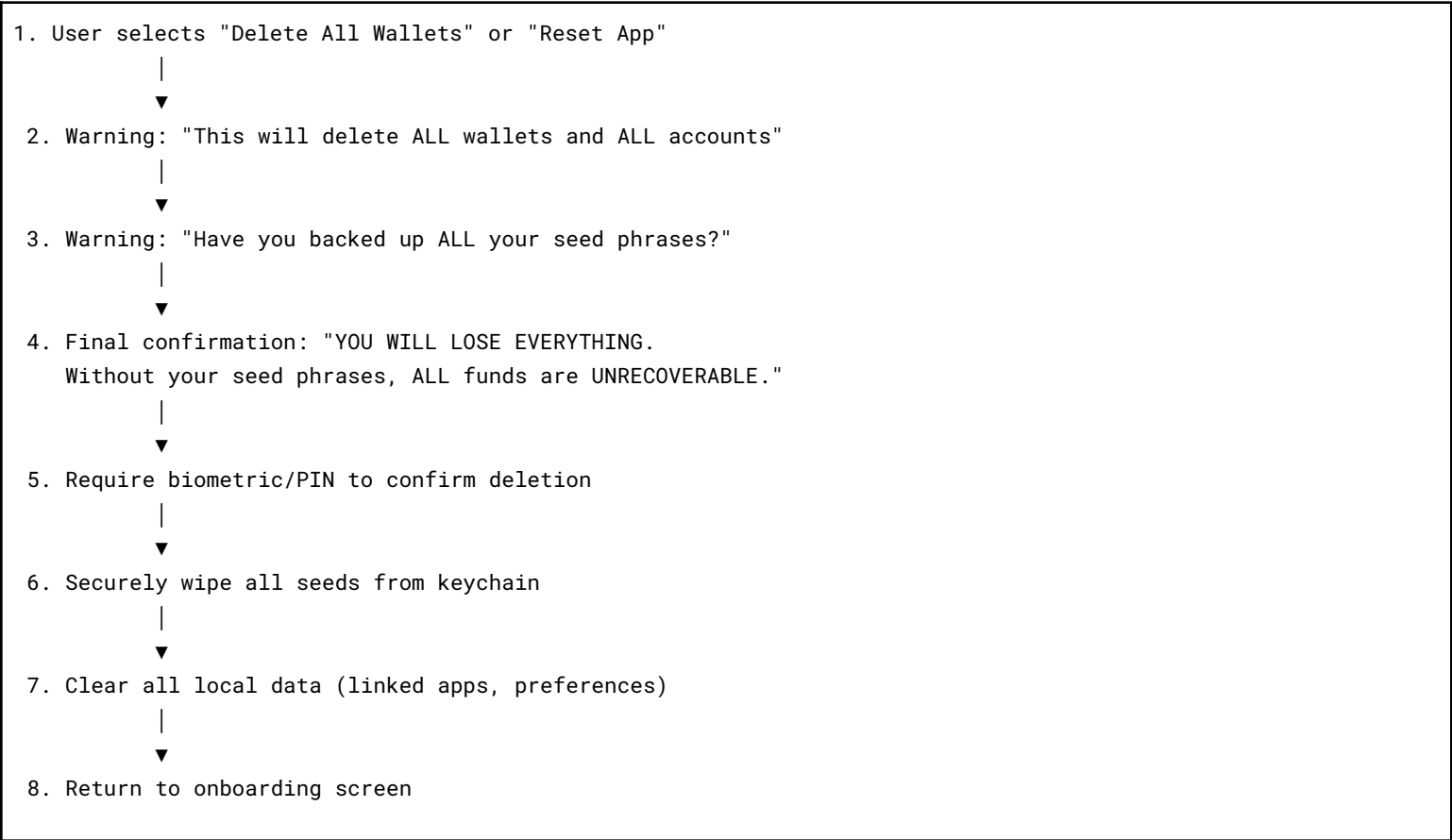


Key Destruction

Delete Single Seed Flow



Delete All Data Flow



Aspect	Requirement	Owner
Confirmation	Multiple explicit warnings before deletion	Mobile Team
Authentication	Biometric/PIN required to confirm	Mobile Team
Secure wipe	Use platform secure deletion APIs	Mobile Team
Granularity	Support deleting single seed or all data	Mobile Team

Key Lost / Compromised

Key rotation is not applicable. Seed phrases are permanent.

Scenario	Action
Seed compromised	User must create new seed and transfer funds from compromised wallet
Lost seed phrase	If platform backup exists, export seed from current device first

Important: Educate users that seed phrase compromise means ALL accounts/addresses derived from that seed are compromised. Other seeds in the app remain safe.

Telemetry & Privacy

Principles

Principle	Description
Privacy First	No PII collected by default.
Transparency	Clear disclosure of what is collected.
Minimal	Collect only what is needed for KPIs.
Backend Only	Data collected from the Bridge

What is Collected ?

- Event Date (When the event occurred)
- **Vault** Version
- **Bridge** Version
- Source (Origin)
- Transaction Hash
- Transaction Size
- Cardano Fee
- Transaction Intent(s) (*Transfer Ada, SC Interactions, etc.*)
- Transaction Inputs/Outputs (*Preparing the prevention system*)
- Transaction State (Accepted, Rejected)
- Event Type
- TBD.

Events:

- Vault Pairing (HTTP POST)
- Unsigned
- Signed
- Unlink
- Link

Retention Policy

Data Type	Retention	Owner
Logs	30 days (Subject to change)	Backend Team
Traces	15 days (Subject to change)	Backend Team
Metrics	1 year (Subject to change)	Backend Team

Note: The retention policy is subject to change due to compliance requirements.

Threats and Security Review

Threats

This document details identified threats, their attack vectors, impact assessments, and required mitigations.

Severity Legend

Severity	Definition
Critical	Total loss of funds or complete system compromise
High	Significant financial loss or unauthorized actions
Medium	Limited exposure, no direct fund loss
Low	Minimal impact, informational exposure

Platform Security Parity Notice

This threat model assumes platform-provided secure storage with cloud sync.

iOS provides turn-key solutions for both seed phrase storage and metadata sync. Whereas Android does not have direct equivalents.

Current Plan for Wallet SDK Development:

The initial Proof-of-Concept (POC) and Minimum Viable Product (MVP) will prioritize the iOS platform for the automated Sync and Recovery features using built-in iCloud Keychain.

For Android, the secret keys will be stored locally on the device. Consequently, in this first version, Android users will be responsible for manually backing up their keys.

List of Identified Threats

List of identified threats, with attack vectors, impacts and mitigations.

Private Key Exfiltration

Description: An attacker gains access to private keys stored on the device or in cloud backup.

Attack Vectors:

Vector	Description	Likelihood
Malware on device	Keylogger or memory scraper captures keys	Medium
Compromised cloud account	Attacker accesses iCloud/Google account	Medium
Physical device theft	Attacker has unlocked device	Low
Backup extraction	Keys extracted from unencrypted backup	Low

Impact: Critical (total loss of funds)

Mitigations:

Mitigation	Owner	Status
Store keys in Keychain / Secured keystore	Mobile Team	Required
Require biometric or PIN to access keys	Mobile Team	Required
Keys only loaded in memory during signing or seed reveal, cleared immediately after	Mobile Team	Required
Never log, debug, or print key material in any environment	All Teams	Required
Cloud backup uses platform encryption (iCloud Keychain, etc.)	Platform (Apple/Google)	Inherited

Refer to official Apple documentation in regards of how the data is handled when using iCloud Keychain:

<https://support.apple.com/en-us/102651>

Unauthorized Transaction Signing

Description: An attacker triggers transaction signing without legitimate user intent.

Attack Vectors:

Vector	Description	Likelihood
Stolen Session	Attacker obtains Session from browser storage, sends signing requests	Medium
Compromised dApp	Malicious dApp sends TX that looks legitimate but drains wallet	Medium
Session hijacking	Attacker takes over linked session	Low

Impact: Critical (loss of funds via signed malicious transaction)

Mitigations:

Mitigation	Owner	Status
Require biometric/PIN confirmation for every signature	Mobile Team	Required
Display clear TX details (IN/OUT, recipient, amount) before signing	Mobile Team	Required
Rate limit signing requests (e.g., max 1 TX per N seconds)	Backend Team	Required
Session rotation	Backend Team	Required
User can reject/cancel any TX from <i>Vault</i>	Mobile Team	Required

Telemetry Data Leakage

Description: Telemetry or logging inadvertently captures sensitive data that could identify users or expose transaction patterns.

Attack Vectors:

Vector	Description	Likelihood
Accidental PII logging	Developer logs user IP or device identifiers	Medium
Telemetry correlation	Combining metrics to profile users	Medium
Third-party telemetry breach	External telemetry provider is compromised	Low
Log file exposure	Logs stored insecurely or accessible publicly	Low

Impact: Medium (privacy breach, potential targeted attacks)

Mitigations:

Mitigation	Owner	Status
No PII in telemetry	All Teams	Required
Hash IP addresses for rate limiting / attack detection	Backend Team	Required
Define and enforce retention policies	Backend Team	Required
Code review checklist: verify no sensitive data in logs	All Teams	Required
Use trusted telemetry stack	Backend Team	Recommended

Cloud Storage Compromise

Description: An attacker gains access to cloud-stored wallet data (seed phrase backup).

Attack Vectors:

Vector	Description	Likelihood
Compromised cloud account	Attacker gains access to user’s Cloud account	Medium
Cloud provider breach	Cloud Infrastructure compromised	Very Low
Weak account security	User has no 2FA, weak password	Medium
Social engineering	Attacker tricks user into revealing cloud credentials	Medium

Impact: Critical (seed phrase exposed = total loss of funds)

Mitigations:

Mitigation	Owner	Status
Use platform keychain with encryption	Mobile Team	Required
Seed phrase encrypted before cloud sync	Platform (Apple iCloud Keychain)	Inherited
Display warning that no one will ever request seed phrase (not even for support, debugging, testing, or validation)	Mobile Team	Required
Encourage users to enable 2FA on cloud accounts	Mobile Team (UX)	Recommended
Display security tips during onboarding	Mobile Team	Recommended

Man-in-the-Middle Attacks

Description: An attacker intercepts communications between components.

Attack Vectors:

Vector	Description	Likelihood
TLS stripping	Downgrade HTTPS to HTTP	Low
Compromised CA	Attacker obtains fraudulent certificate	Very Low
DNS hijacking	Redirect Bridge/API domain to attacker server	Low
Malicious proxy	Corporate/public WiFi intercepts traffic	Medium
Unsigned TX tampering	Modify TX in transit before signing	Medium

Impact: High (transaction tampering, credential theft)

Mitigations:

Mitigation	Owner	Status
TLS everywhere	All Teams	Required
Enforce authenticated requests	Backend Team	Required
User reviews TX details on wallet before signing	Mobile Team	Required
HSTS headers on Bridge server and Anvil API	Backend Team	Required

Private Key Extraction (Technical Methods)

Description: An attacker uses technical tools to extract private keys from device storage, memory, or backups.

Attack Vectors:

Vector	Description	Likelihood
Memory dump	Extract keys from RAM during/after signing	Low
Debug tools	Attach debugger to running app, inspect memory	Low
Device forensics	Extract data from seized/stolen device	Low
Backup file analysis	Parse unencrypted local backups	Medium
Jailbreak/root exploits	Bypass OS protections to access keychain	Low

Impact: Critical (total loss of funds)

Mitigations:

Mitigation	Owner	Status
Clear keys from memory immediately after use	Mobile Team	Required
Disable sensitive operations in debugger environments (production builds)	Mobile Team	Required
Store seed phrase in platform secure storage with cloud sync (when available)	Mobile Team	Required
Exclude sensitive data from local backups	Mobile Team	Required
Obfuscate release builds	Mobile Team	Recommended

Platform secure storage:

Platform	Storage Method	Cloud Sync	Status
iOS	iCloud Keychain	Native	Ready to use
Android	Encrypted Shared Preferences	Research required	Research

Note for developers: iOS provides a turn-key solution with iCloud Keychain. Android, Linux & Windows do not have a direct equivalent. Developers must research and validate the best approach to achieve similar functionality.

Seed Phrase Compromise Through Telemetry or Logging

Description: Seed phrase accidentally exposed via logs, crash reports, or telemetry.

Attack Vectors:

Vector	Description	Likelihood
Debug logging	Developer accidentally logs seed during testing	Medium
Crash reports	Seed in memory included in crash dump	Low
Third-party SDKs	Analytics/crash SDK captures screen or memory	Low
Console output	Seed printed to console in debug builds	Medium

Impact: Critical (total loss of funds)

Mitigations:

Mitigation	Owner	Status
Never log seed phrase in any environment	All Teams	Required
Redact sensitive fields in crash reports	Mobile Team	Required
Audit third-party SDKs for data collection practices	Mobile Team	Required
Code review checklist: verify no seed/key logging	All Teams	Required
Prevent screenshots/screen recording on seed reveal screen	Mobile Team	Required
Static analysis for sensitive data patterns	All Teams	Recommended

Note on screenshot prevention: iOS and Android both support preventing screenshots on specific screens.

Transaction Tampering or Replay Attacks

Description: Attacker modifies transaction content before signing, or replays a previously signed transaction.

Attack Vectors:

Vector	Description	Likelihood
TX modification on Bridge	Attacker compromises <i>Bridge</i> , modifies unsigned TX	Low
Replay attack	Resubmit previously signed TX to drain funds	Low (blockchain handles)
dApp manipulation	Malicious dApp crafts TX that looks benign but drains wallet	Medium

Impact: Critical (loss of funds)

Mitigations:

Mitigation	Owner	Status
User must review TX details before signing	Mobile Team	Required
Display human-readable TX summary (IN -> OUT, who gets what)	Mobile Team	Required
Cardano blockchain uses UTxO model (natural replay protection)	Blockchain	Inherited
TX has TTL (expires after slot)	Blockchain	Inherited

TX Display Requirements:

This wallet is designed for everyone. The transaction review screen must:

Mode	Description	Default
Simple view	Friendly summary: what you send (IN), what you receive (OUT), who gets what	Yes

Phishing Attacks Mimicking Wallet Interface

Description: Attacker creates fake wallet interface (fake app, fake linking page, fake dApp) to steal credentials or trick users into signing malicious transactions.

Attack Vectors:

Vector	Description	Likelihood
Fake mobile app	Attacker publishes lookalike app on app store	Medium
Fake linking page	Phishing site mimics <i>Client</i> UI (Web Component / Weld Integration)	Medium
Fake dApp	Malicious dApp pretends to be legitimate service	High
Social engineering	Attacker contacts user pretending to be support	Medium

Impact: Critical (seed phrase theft, malicious transaction signing)

Mitigations:

Mitigation	Owner	Status
Official app only from verified app stores	Mobile Team	Required
Educate users: Anvil will never ask for seed phrase	All Teams	Required
Clear branding in wallet to distinguish from fakes	Mobile Team	Required
Signing always happens in official wallet app (never in <i>client/browser</i>)	Architecture	Inherited

Supply Chain Compromise of Wallet Dependencies

Description: Attacker compromises a third-party library or dependency used by the **Vault**, SDK(s), or **Bridge** to inject malicious code.

Attack Vectors:

Vector	Description	Likelihood
Compromised package	Malicious code in dependency update	Medium
Typosquatting	Developer installs similarly-named malicious package	Low
Compromised CI/CD	Attacker injects code during build process	Low
Abandoned dependency takeover	Attacker takes over unmaintained package	Low

Impact: Critical (could exfiltrate keys, modify transactions)

Mitigations:

Mitigation	Owner	Status
Lock dependency versions	All Teams	Required
Regular dependency audits	All Teams	Required
Minimize dependencies, especially for crypto operations	All Teams	Required
Review dependency changes in PRs	All Teams	Required
Use Dependabot or similar for security alerts	All Teams	Recommended
Sign releases and verify signatures	All Teams	Recommended

Cloud Backup Failure

Description: User’s seed phrase backup fails silently, leading to total loss of funds when they need to recover on a new device.

Attack Vectors:

Vector	Description	Likelihood
Silent backup failure	Cloud sync fails without user notification	Medium
Platform account issues	User’s Cloud account has storage or sync issues	Low
Backup corruption	Backup data becomes corrupted during sync	Low
Account migration	User changes platform account, backup not transferred	Medium

Impact: Critical (total loss of funds if device is lost/replaced)

Mitigations:

Mitigation	Owner	Status
Verify backup exists after wallet creation	Mobile Team	Required
Display backup status indicator in wallet settings	Mobile Team	Required
Prompt user to verify backup periodically	Mobile Team	Recommended
Provide manual backup option (view seed phrase) as fallback	Mobile Team	Required
Warn user if backup verification fails	Mobile Team	Required

Note: Users should be encouraged to verify their backup works by confirming they can access their seed phrase from the wallet. The wallet should make this easy to check without exposing the seed phrase unnecessarily.

Seed Phrase Theft During Recovery

Description: Attacker tricks users into revealing their seed phrase during the recovery process on a new device, through social engineering or fake recovery flows.

Attack Vectors:

Vector	Description	Likelihood
Fake recovery app	User downloads fake wallet app that harvests seed phrase	Medium
Phishing during recovery	Attacker sends fake “verify your wallet” emails/messages	High
Fake support	Attacker poses as Anvil support requesting seed for recovery	Medium
Malicious website	Fake recovery page asks user to enter seed phrase	High

Impact: Critical (total loss of funds)

Mitigations:

Mitigation	Owner	Status
Educate users during onboarding: seed phrase is like a password	Mobile Team	Required
Display warning: no one will ever ask for seed phrase (see list below)	Mobile Team	Required
Recovery only happens in official app downloaded from verified app store	Mobile Team	Required
Seed phrase entry screen shows security warnings	Mobile Team	Required
No seed phrase entry via <i>Client</i> / Web Browser (<i>Vault</i> only)	Architecture	Inherited

User education points:

- Your seed phrase controls your funds. Anyone with it can steal everything.
- **No one will ever ask for your seed phrase. Not Anvil. Not Apple. Not Google. Not Microsoft. Not support. Not for recovery. Not for verification. Not for debugging. Never.**
- Only enter your seed phrase in the official Anvil wallet app.
- If anyone asks for your seed phrase, it is a scam. Report it.

Trust Boundaries

Trust boundaries define where validation and verification must happen. Data crossing a boundary should not be trusted without verification.

Component Clarification

Component	Purpose	Lifetime
Linking Web Component	Displays PIN for wallet linking flow	Temporary
Weld SDK	CIP-30 compliant API, handles all wallet interactions	Persistent (runs in dApp context)

The *Linking Web Component* is only used during the initial wallet linking. After linking, the **Client** reads the Session Token from browser storage and communicates directly with the **Bridge** for all CIP-30 operations.

The **Client** includes the custom code to interact with the **Bridge**, this code should be added as a “plugin”, so it can be used outside *Weld* as well.

Boundary Definitions

From	To	Validation Required
Client (dApp, etc.)	SDK	SDK runs in dApp context (In the <code>window.cardano</code>); dApp must not tamper with Session Token
Client (SDK)	Bridge	Session Token authentication; TLS; rate limiting
Bridge	Vault	Session Token authentication

What Each Zone Trusts

Zone	Trusts	Does NOT Trust
Vault (Trusted)	Platform keychain, user input	Bridge content; unsigned TX (user must manually review)
Bridge (Controlled)	Valid Session Token holders	TX content; dApp identity beyond Session Token
SDK (Semi-trusted)	Bridge responses (over TLS)	dApp code (Session Token stored in browser, dApp could access)
Client (Untrusted)	Nothing assumed	Must authenticate via Session Token to Bridge

Critical Trust Boundary: User Review

The most critical trust boundary is the **user themselves**. The wallet displays transaction details (IN/OUT) and the user must verify before signing. No technical control can prevent a user from signing a malicious transaction if they approve it.

Wallet responsibility: Display clear, human-readable transaction summaries.

Risk Matrix

This section provides a summary view of all identified threats for prioritization.

Priority Definitions

Priority	Definition	Action
P1	Critical risk, must address before launch	Required mitigations, no exceptions
P2	High risk, should address before launch	Required mitigations, exceptions need approval
P3	Medium risk, address in early releases	Recommended mitigations
P4	Low risk, address when convenient	Optional mitigations

Threat Summary

Threat	Impact	Likelihood	Priority	Primary Owner
Private Key Exfiltration	Critical	Medium	P1	Mobile Team
Unauthorized Transaction Signing	Critical	Medium	P1	Mobile + Backend
Telemetry Data Leakage	Medium	Medium	P3	All Teams
Cloud Storage Compromise	Critical	Medium	P1	Platform (inherited)
Man-in-the-Middle Attacks	High	Low-Medium	P2	Backend Team
Private Key Extraction (Technical)	Critical	Low	P2	Mobile Team
Seed Phrase in Telemetry/Logs	Critical	Medium	P1	All Teams
Transaction Tampering/Replay	Critical	Medium	P1	Mobile + Blockchain
Phishing Attacks	Critical	Medium-High	P1	All Teams
Supply Chain Compromise	Critical	Medium	P1	All Teams
Cloud Backup Failure	Critical	Medium	P1	Mobile Team
Seed Phrase Theft During Recovery	Critical	High	P1	Mobile Team

Accepted Risks

Risk	Rationale
Jailbroken/rooted devices	User responsibility; Anvil not liable
Cloud provider breach	Extremely low likelihood; trust Cloud Providers
User approves malicious TX	Cannot prevent; mitigate with clear TX display

Architecture Decisions

Our Proof of Concept will be developed using the following core technologies:

Vault

- **Mobile Development:** Flutter
- **Secure Storage:** flutter_secure_storage
- **Wallet & Transaction Management:**
 - cbor
 - catalyst_key_derivation
 - catalyst_cardano_serialization
 - bip39
- **Targeted Platforms:** iOS and Android
- **Authentication Partner(s):**
 - **Apple:** A fully functional solution leveraging the Apple ecosystem is the primary focus for this concept. Cloud synchronization is managed using the built-in *Apple iCloud Keychain*.
 - Reference: <https://support.apple.com/en-us/109016>
 - **Android:** Given the absence of a native, built-in solution, the development team will explore various options. The goal is to prioritize the use of *Google's native solutions* where available.
 - The primary goal of authentication is to provide a secure semi-custodial wallet that includes a recovery solution. Because this setup requires that the keys remain solely in the users' possession and accessible only by them, *Apple iCloud Keychain* is an ideal choice for our implementation.

Bridge

- **Web Server:** Gin
- **Protocol:** Websocket
- **Database:** Redis
- **Cloud Provider:** AWS

Client

- **Role(s):**
 - The UI Component connects with the **Bridge** and implements CIP-30 compliant functions. This architecture eliminates the need to store or handle sensitive data in the browser (or any implementing application), creating a security layer by separating dApp interactions from the signing process.
- **SDK: Modular Web Component** (Future-ready integration)
 - The SDK prioritizes reusability and customization, allowing dApps to adapt the implementation to their unique UI/UX needs.
- **Integration:**
 - **Standalone Module:** Developers install a dedicated package that's part of Weld, our open-source Wallet Connector (<https://github.com/Cardano-Forge/weld>)
 - **Weld Integration:** For dApps already using Weld, the bindings and integration are included automatically.
- **Research & Investigation:**
 - **Browser Extension:** A solution that exclusively communicates with the **Bridge** without storing private data in the browser. This approach enables seamless integration, provided the dApp scans for all wallets installed in the user's browser.

Milestone 2

- ☒ ~~Link to Video showing transaction signing, send/receive transactions on test, and signal flows/tests~~
- ☒ ~~Link to Signed Transaction~~
- ☒ ~~Link to Video showing demo functionality~~
- ☒ ~~Link to Test Results~~

Manual tests results

Status	Description	Note
PASS	Test error handling when device has biometric disabled on Apple	Returns expected user error
PASS	Test error handling when device has biometric disabled on Android	Returns expected user error
PASS	Test key persistence on Apple after uninstalling and reinstalling	Keys persist after reinstallation
N/A	Test key persistence on Android after uninstalling and reinstalling	As mentioned above, Google ecosystem does not have a turnkey solution.
PASS	Test to validate failed auth	We use builtin Apple and Android solutions.
PASS	Test key persistence on Apple after app restart	Keys are safely stored on device
PASS	Test key persistence on Android after app restart	Keys are safely stored on device

Automated test results on Android

Class io.hodei.vault.MainActivityTest

all > [io.hodei.vault](#) > MainActivityTest

5
tests

0
failures

0
skipped

43.310s
duration

100%
successful

Tests

Test	Medium_Phone_API_36(AVD) - 16
runDartTest[secure_storage_test canceling auth dialog throws exception]	passed (7.342s)
runDartTest[secure_storage_test multiple wallet mnemonics persist with different suffixes]	passed (22.689s)
runDartTest[secure_storage_test vault mnemonic persists across SecureStorage instances]	passed (0.915s)
runDartTest[secure_storage_test wallet list JSON persists across instances]	passed (0.674s)
runDartTest[secure_storage_test wallet read requires device authentication]	passed (11.690s)

Live Demos

- <https://anvil-wayup-git-task-hodei-cardano-forge.vercel.app/>
 - Can interact with Wayup Marketplace
- <https://hodei.io>
 - Can test pool delegation & sign data

Links to Video and Submitted Transaction

- Submitted Transaction:
<https://cardanoscan.io/transaction/4bb85645a633bf4f6015febd5d3db07c23153c7d8bb10d8f8564c61fb91bc039?tab=utxo>
- <https://youtu.be/E6v9UEtI2OY>

Wallet Created on iPhone and secrets stored in Apple icloud and accessible on MacOS Keychain:

1

```
[{"id":"a1c074575674cc6394fd979c0c5f989e9724d1edf585e893b06826d99cb22e4e","name":"test","accounts":[{"id":0,"name":"Account #0","paymentPub":"396c3ec478d9de6492177bb3df5eb85b9033447ab4a9ae7fc2e7ae7a31e9a137","stakePub":"493515e695d5709633d607d4244598d6d1249115605a79925a9e24ab94203b7e"}]}
```

Keychain Access

All Items

Passwords

Secure Notes

My Certificates

Keys

Certificates

io.hodel.vault

Kind: application password

Account: walletList

Where: io.hodel.vault

Modified: Today, 2:23 PM

Name	Kind	Date Modified	Expires	Keychain
io.hodel.vault	application password	Today, 2:23 PM	--	iCloud
io.hodel.vault	application password	Today, 2:23 PM	--	iCloud
io.hodel.vault	application password	Today, 2:23 PM	--	iCloud
io.hodel.vault.hello	application password	Today, 2:03 PM	--	iCloud
io.hodel.vault.hello	application password	Today, 2:03 PM	--	iCloud
io.hodel.vault.hello	application password	Today, 2:03 PM	--	iCloud

io.hodel.vault

Attributes

Access Control

Name: io.hodel.vault

Kind: application password

Account: walletList

Where: io.hodel.vault

Comments:

Show password: [{"id":"a1c074575674cc6394fd979c0c5f98"}]

Show Changes

2:24

test

Account #0

0.00 tA

Base

addr_test1qz...hs76t6ks

Stake

stake_test1u...tcg4r7j5

Sessions

No sessions

Signature Requests

No requests

Mainnet

Preprod

Delete All Keychain

Milestone 3

<https://milestones.projectcatalyst.io/projects/1400011/milestones/3>

Privacy impact assessment

- [Hodei Wallet Privacy Impact Assessment](#) (Requires \$Cash - CEO approval)

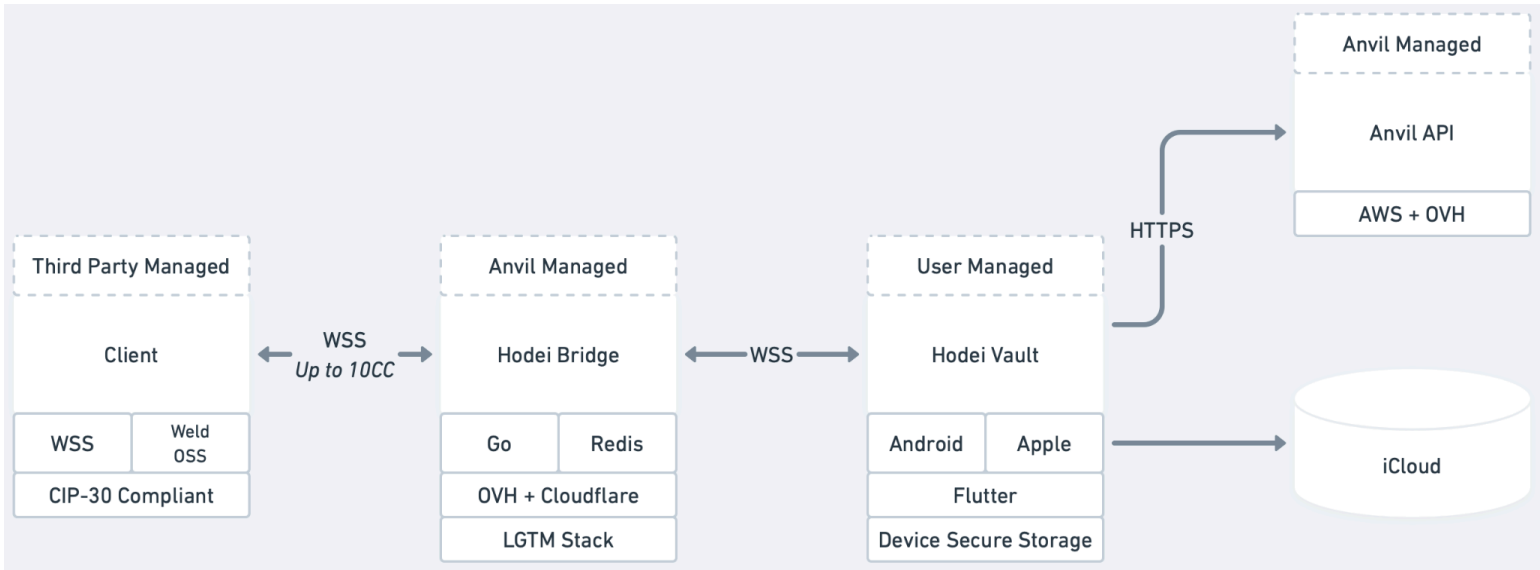
Proofs

-  Hodei Wallet in 100 Seconds - One Wallet for Every Platform
- [Android Signing Flow](#)
- [Autosync and Switch Devices](#)
 - The recovery is automated by Apple Keychain through iCloud.
- [iPhone Signing Flow](#)
- [MacOS Signing Flow](#)
- **Hodei Wallet Chrome Extension** making it easier to dApp to integrate
 - <https://chromewebstore.google.com/detail/hodei/hjnmkeighdionceaehneaebloakklojj>
- Tests: https://youtu.be/IZA6_I_I9gw
- Knight Guilds
 - <https://cardanoscan.io/transaction/f2a854f9d579dbb38f44cc4c87c00d610f06f8acea79e4508f5f520393be7108?tab=summary>
 - <https://cardanoscan.io/transaction/25d10fdb6ca33ef6e0ddb4c135772a92b27e9ef3261e606e6e0f5cdd103f79e3>
 - <https://youtu.be/sdwT9Yw4IYg>

Telemetry Update

- Bridge Logging:
 - No PII, no IP, we use cloudflare to protect the servers.
- Bridge Metrics:
 - We track the message type to tell us the overall stability of the platform and be proactive in case of issues. No User information is added to those metrics.
- Bridge Traces:
 - IPs are truncated to anonymize it.
- Bridge Database:
 - No PII are stored in the database.
- Vault:
 - No third-party monitoring, error tracking, logger, etc.
 - Seedphrases and Private keys are stored on the device, and can be stored in iCloud (when available)
- Hodei Client:
 - No third-party monitoring, error tracking, logger, etc.

Infrastructure



Test Report Milestone 3

Our main focus has been manual testing to identify, report, and fix all timing issues.

The team tackled ~10 tickets covering QOL, bugs, and UX/UI improvements (see changelog for details), along with testing the Weld and Hodei Client integrations on Wayup.io and the demo Hodei.io.

On the automation side, we are running the same test sequence from Milestone 2 to ensure no regressions.

Status	Description	Note
PASS	Switch Device on Apple	
PASS	Signing flow on iPhone	
PASS	Signing flow on MacOS	
PASS	Signing flow on Android	
PASS	Autosync keychain (apple only)	Takes 30 seconds + time for iCloud to sync
PASS	Validate websocket stability on integrations	Tested Wayup and Hodei
FAIL	No more deadlock	Missing pin modal on Wayup
PASS	Seedphrase Recovery on apple	
PASS	Seedphrase Recovery on android	
PASS	Consent message appears once and block the app until accepted	
PASS	Recovery system on Apple	Requires iCloud Keychain
N/A	Recovery system on Android	

PASS	CIP-30 integration and support	
PASS	CIP-8 Signature	

tmux_sessionizer

08:56 [5/20]

```
RUN v4.0.18 /Users/dodi/Documents/intek/hodei-client

✓ src/lib.test.ts > initialize > creates window.cardano.hodei with correct shape 22ms
✓ src/lib.test.ts > initialize > returns same instance on repeated calls (singleton) 0ms
✓ src/lib.test.ts > wallet API (paired) > getNetworkId returns 1 for mainnet 53ms
✓ src/bridge.test.ts > checkToken > returns valid on 200 1ms
✓ src/bridge.test.ts > checkToken > returns notFound on 404 0ms
✓ src/bridge.test.ts > checkToken > returns tooManyConnections on 409 0ms
✓ src/bridge.test.ts > checkToken > sends correct Authorization header 0ms
✓ src/bridge.test.ts > Bridge.connect > converts http to ws in URL and appends /client/ws 51ms
✓ src/bridge.test.ts > Bridge.connect > connects without token param when no stored token 51ms
✓ src/bridge.test.ts > Bridge.connect > appends token param when stored token is valid 52ms
✓ src/bridge.test.ts > Bridge.connect > throws when stored token returns 409 (too many connections) 1ms
✓ src/lib.test.ts > wallet API (paired) > getNetworkId returns 0 for preprod 52ms
✓ src/lib.test.ts > wallet API (paired) > getUsedAddresses returns [baseAddress] 52ms
✓ src/bridge.test.ts > Bridge.connect > deletes token and connects without param when token returns 404 51ms
✓ src/bridge.test.ts > Bridge.connect > resolves with pairing state and calls onStateChange 55ms
✓ src/lib.test.ts > wallet API (paired) > getUnusedAddresses returns [] 50ms
✓ src/lib.test.ts > wallet API (paired) > getChangeAddress returns baseAddress 53ms
✓ src/bridge.test.ts > Bridge.connect > resolves with paired state 52ms
✓ src/bridge.test.ts > Bridge.connect > rejects on invalid first message 53ms
✓ src/lib.test.ts > wallet API (paired) > getRewardAddresses returns [stakeAddress] 53ms
✓ src/lib.test.ts > wallet API (paired) > getUtxos delegates to anvil 55ms
✓ src/bridge.test.ts > post-connection messages > wallet_updated merges payload keeping sessionId/token 53ms
✓ src/bridge.test.ts > post-connection messages > ignores invalid message types without crashing 53ms
✓ src/lib.test.ts > wallet API (paired) > getBalance delegates to anvil 54ms
✓ src/lib.test.ts > wallet API (paired) > submitTx delegates to anvil 56ms
✓ src/bridge.test.ts > disconnection > disconnect() closes WS with code 1000 51ms
✓ src/bridge.test.ts > disconnection > close event code 4001 deletes stored token 54ms
✓ src/lib.test.ts > wallet API (not paired) > all methods throw with code -3 when not paired 55ms
✓ src/lib.test.ts > signTx > sends sig_req_created message and resolves on sig_req_accepted 61ms
✓ src/bridge.test.ts > disconnection > close event code 1000 sets state to closed and calls onStateChange 51ms
✓ src/bridge.test.ts > reconnection > non-1000 close triggers reconnect 4ms
✓ src/bridge.test.ts > reconnection > exponential backoff: 0s, 2s, 4s, 8s, 16s, 32s 3ms
✓ src/bridge.test.ts > reconnection > max 5 attempts then continues reconnecting with same delay 3ms
✓ src/bridge.test.ts > send > serializes message to JSON and calls ws.send 52ms
✓ src/bridge.test.ts > send > no-op when not connected 0ms
✓ src/bridge.test.ts > isBridgeState > accepts status pairing 0ms
✓ src/bridge.test.ts > isBridgeState > accepts status paired 0ms
✓ src/bridge.test.ts > isBridgeState > accepts status closed 0ms
✓ src/bridge.test.ts > isBridgeState > accepts status error 0ms
✓ src/bridge.test.ts > isBridgeState > rejects invalid shapes 0ms
✓ src/bridge.test.ts > assertSigReqResponse > accepts sig_req_accepted 0ms
✓ src/bridge.test.ts > assertSigReqResponse > accepts sig_req_rejected 0ms
✓ src/bridge.test.ts > assertSigReqResponse > throws on anything else 0ms
✓ src/lib.test.ts > signTx > rejects on sig_req_rejected 53ms
✓ src/lib.test.ts > signTx > ignores non-matching requestId 52ms
✓ src/lib.test.ts > signData > splits accepted response on :: into {signature, key} 51ms
✓ src/lib.test.ts > signData > throws internalError (code -2) if response has no :: 53ms
✓ src/lib.test.ts > isEnabled > returns true when bridge is connected 53ms
✓ src/lib.test.ts > isEnabled > returns false when no token and not connected 2ms
✓ src/lib.test.ts > isEnabled > delegates to checkToken when token exists but not yet enabled 1ms
✓ src/lib.test.ts > isEnabled > returns false when checkToken returns invalid 2ms
✓ src/lib.test.ts > command handling > dialog_closed while pairing disconnects bridge 55ms
✓ src/lib.test.ts > command handling > disconnected command disconnects bridge 55ms
✓ src/lib.test.ts > command handling > unlinked command sends unlink message over WS 55ms

Test Files 2 passed (2)
Tests 54 passed (54)
Start at 08:56:15
Duration 1.34s (transform 114ms, setup 0ms, import 128ms, tests 1.75s, environment 250ms)
[hodei-cli2:nvim- 3:[tmux]]*
```

"MacBookPro" 08:57 27-Apr-26

Changelog

2026-04-02

New

- Concurrent client connections
- Consent message on startup
- Seedphrase recovery
- Word suggestions when restoring
- Ability to paste your seedphrase

Fixes

- Synchronization issues on iCloud
- Conflicts with browser extensions
- UI glitches in pin and seedphrase inputs
- 4 additional bug fixes and quality of life improvements

2026-04-10

Fixes

- Device Syncing on Apple Ecosystem
- Accept or deny a signature from a vault app on the same device that made the request (switch between browser and Hodei apps)

2026-04-17

QOL

- Improve signature flow when using a dApp from the same device as the vault
- Make web socket connections more seamless
- Reconnect on vault or client focus

Fixes

- Fix dead lock state when two devices try to connect using the same vault credentials
- Cleanup expired signature requests on client reconnect